# Blockchain Experiment 8

**Aim:** To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

**Theory:**

1. Blockchain Technology

A blockchain is a distributed, decentralized, and immutable ledger that records transactions across a network of computers. Each record (called a block) contains a cryptographic hash of the previous block, a timestamp, and transaction data, forming a chain. Because every participant holds a copy of the ledger and any alteration to a block invalidates all subsequent blocks, the data stored on a blockchain is highly tamper-resistant and transparent.

Key properties of blockchain:

• Decentralization: No single entity controls the network; authority is distributed across nodes. •

Immutability: Once data is written to the blockchain, it cannot be altered or deleted. •

Transparency: All transactions are visible to network participants.

• Consensus: Nodes agree on the state of the ledger through consensus mechanisms (e.g., Proof of Work, Proof of Stake).

• Security: Cryptographic hashing (SHA-256) ensures data integrity.

2. Ethereum & Smart Contracts

Ethereum is an open-source, decentralized blockchain platform that supports programmable transactions through Smart Contracts. A smart contract is a self-executing program stored on the blockchain whose terms are written directly into code. Once deployed, a smart contract runs automatically when predefined conditions are met, without the need for intermediaries.

Smart contracts in Ethereum are written in Solidity, a statically-typed, contract-oriented programming language. They are compiled into bytecode and deployed to the Ethereum Virtual Machine (EVM),which executes the contract logic on every node in the network.

Key concepts:

• Gas: A unit measuring the computational effort required to execute operations. Users pay gas fees in ETH to incentivize miners/validators.

• ABI (Application Binary Interface): Defines how to interact with the deployed contract — the

function signatures and data encodings.

• Address: Every deployed contract gets a unique Ethereum address on the blockchain.

• msg.sender: A global variable in Solidity that holds the address of the account that called the

current function.

3. Ganache

Ganache (by Truffle Suite / Consensys) is a personal blockchain for Ethereum development. It simulates a full Ethereum node locally on your machine, providing:

• 10 pre-funded test accounts, each loaded with 100 ETH.

• Instant mining — transactions are confirmed immediately (no wait time).

• A local RPC server (default: http://127.0.0.1:7545) that tools like MetaMask and Remix can

connect to.

• A GUI showing real-time accounts, blocks, transactions, and logs for easy inspection.

Ganache is used exclusively for development and testing. It allows developers to deploy and test

contracts without spending real ETH or waiting for mainnet/testnet confirmations.

4. MetaMask

MetaMask is a cryptocurrency wallet and browser extension that acts as a bridge between a web browser and the Ethereum blockchain. It allows users to manage Ethereum accounts, sign transactions, and interact with decentralized applications (dApps) directly from the browser. In development, MetaMask is configured to point to a local Ganache network instead of the Ethereum mainnet, enabling safe testing with dummy ETH.

5. Remix IDE

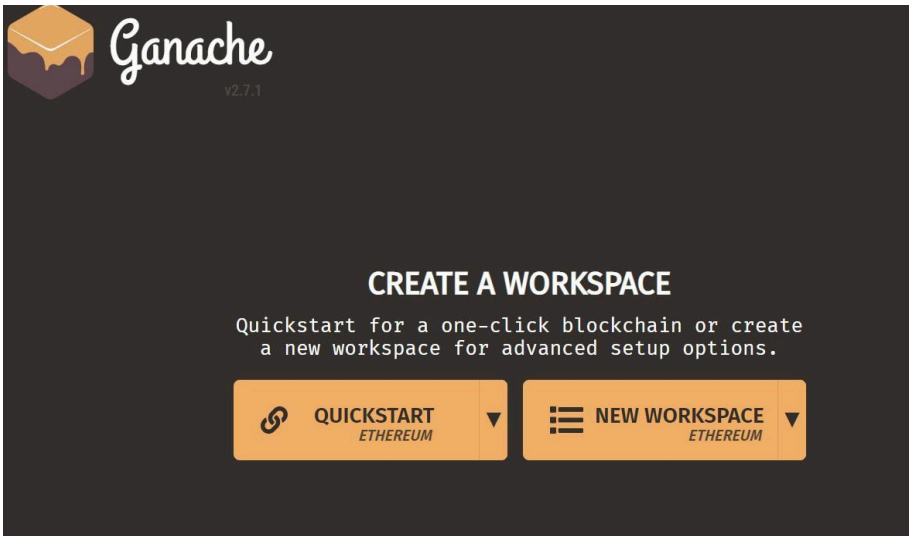
Remix IDE is a web-based integrated development environment for Solidity smart contract development, available at remix.ethereum.org. It provides a code editor, Solidity compiler, contract deployer, and debugger in a single interface. Remix can connect to various environments including the built-in JavaScript VM, Hardhat, or an Injected Provider (MetaMask), the last of which routes deployment through MetaMask to whichever network (Ganache, testnet, or mainnet) MetaMask is currently connected to.

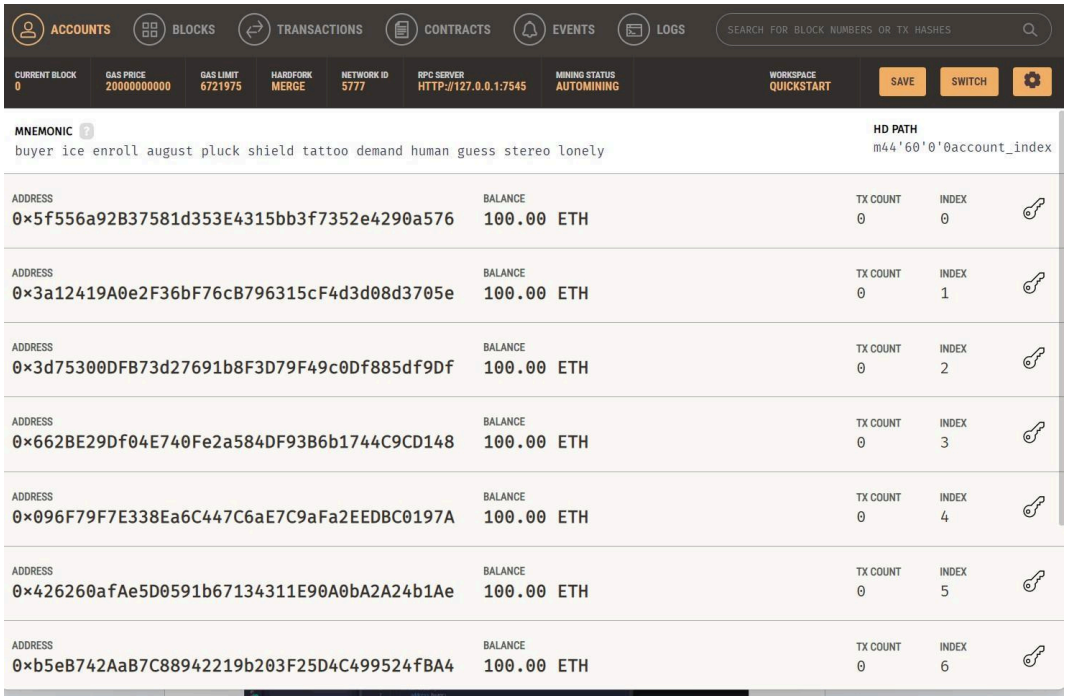
6. Supply Chain Use Case

A supply chain smart contract automates the tracking of goods as they move between parties — in this experiment, a pharmaceutical product (medicine) passing through four stages: Manufacturer, Distributor, Retailer, and Customer. Each stage updates the contract state on-chain, creating a transparent and tamper-proof audit trail. This eliminates the need for a trusted central authority to verify ownership transfers and ensures all parties see the same data

**Implementation:**

Step 1:Install Ganache and click on Quick Start Ethereum






Note the following details from Ganache for later use:

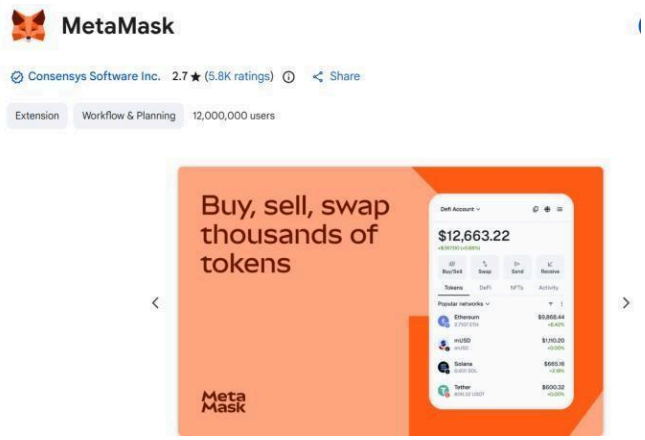
• RPC Server URL: http://127.0.0.1:7545

• Network ID: 5777

• Private keys of accounts (click the key icon next to each address)

Step 2: Install & Setup MetaMask

MetaMask is a browser extension that serves as an Ethereum wallet and allows interaction with decentralized applications. Open Google Chrome and go to the Chrome Web Store.Search for MetaMask (by Consensys Software Inc.) and click Add to Chrome.




After installation, click 'Create a new wallet' on the MetaMask welcome screen. Set a password and save the Secret Recovery Phrase securely

Add Ganache Network to MetaMask

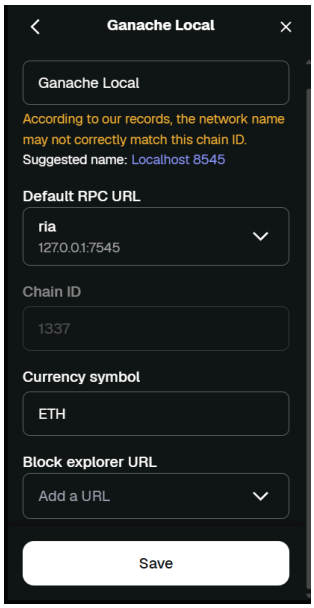
9. In MetaMask, click the network dropdown and select 'Add a custom network'.

10. Enter the following details for the RPC

URL: RPC URL: http://127.0.0.1:7545

RPC Name: Ganache Local

11. Click Save

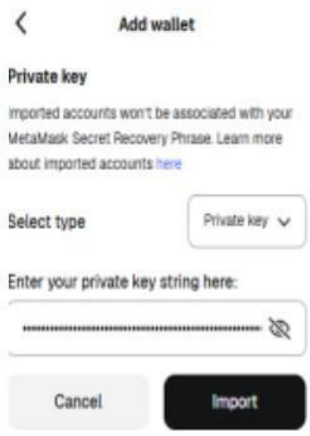



Import Ganache Accounts into MetaMask

In Ganache, click the key icon next to an account address to reveal its private key.

In MetaMask, go to Accounts > Add wallet > Import using Private Key.

Paste the private key and click Import.




Create the Smart Contract in Remix IDE

Remix IDE is a web-based Solidity development environment at remix.ethereum.org.

Open remix.ethereum.org in the browser.

In the File Explorer, create a new file named DmartRetail.sol.

Write the following smart contract:

```
// SPDX-License-Identifier:
MIT pragma solidity ^0.8.0;
contract DmartRetail {
struct Order {
address buyer;
string productIds;
uint totalAmount;
uint timestamp;
}
struct Review {
string productId;
address reviewer;
uint8 rating;
}
Order[] public orders;
Review[] public reviews;
mapping(string => mapping(address => bool)) public reviewed;
// Place Order
function placeOrder(string memory _products, uint _amount) public {
orders.push(Order(msg.sender, _products, _amount, block.timestamp));
}
// Add Review
```

```
function addReview(string memory _productId, uint8 _rating) public {
require(_rating >= 1 && _rating <= 5, "Invalid rating");
require(!reviewed[_productId][msg.sender], "Already reviewed");
reviews.push(Review(_productId, msg.sender, _rating));
reviewed[_productId][msg.sender] = true;
}
// Get total orders
function getOrderCount() public view returns(uint) {
return orders.length;
}
// Get total reviews
function getReviewCount() public view returns(uint) {
return reviews.length;
}
}
```

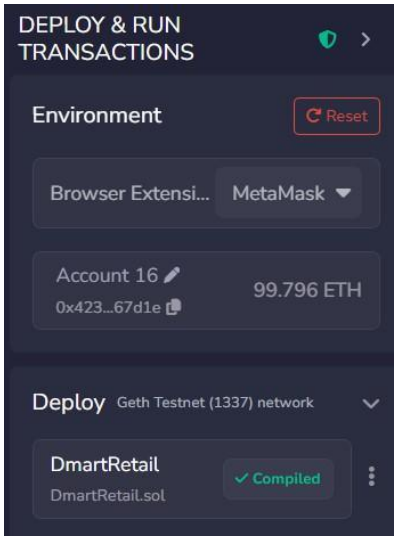
Compile the Smart Contract

In Remix, click the Solidity Compiler icon on the left sidebar.

Select compiler version 0.8.x.

Click Compile SupplyChain.sol.

A green checkmark confirms successful compilation.

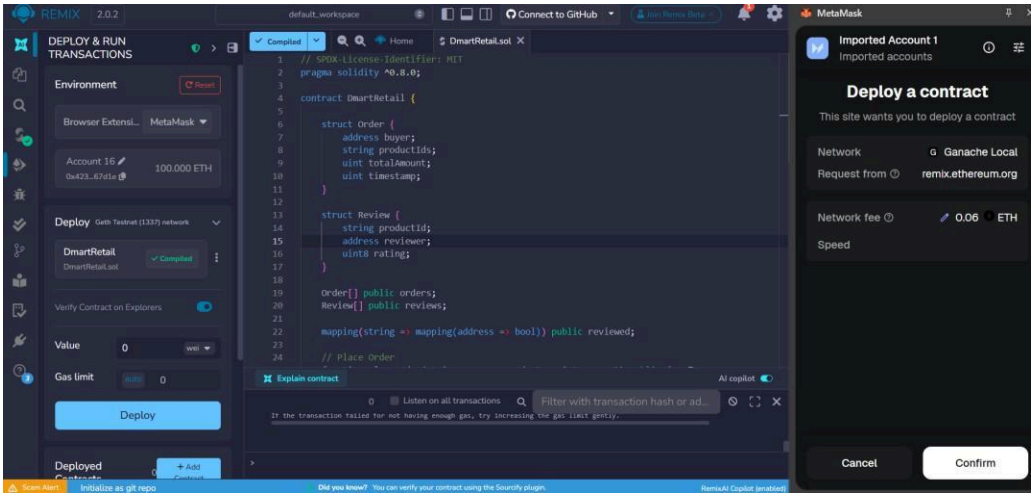



Deploy the Smart Contract

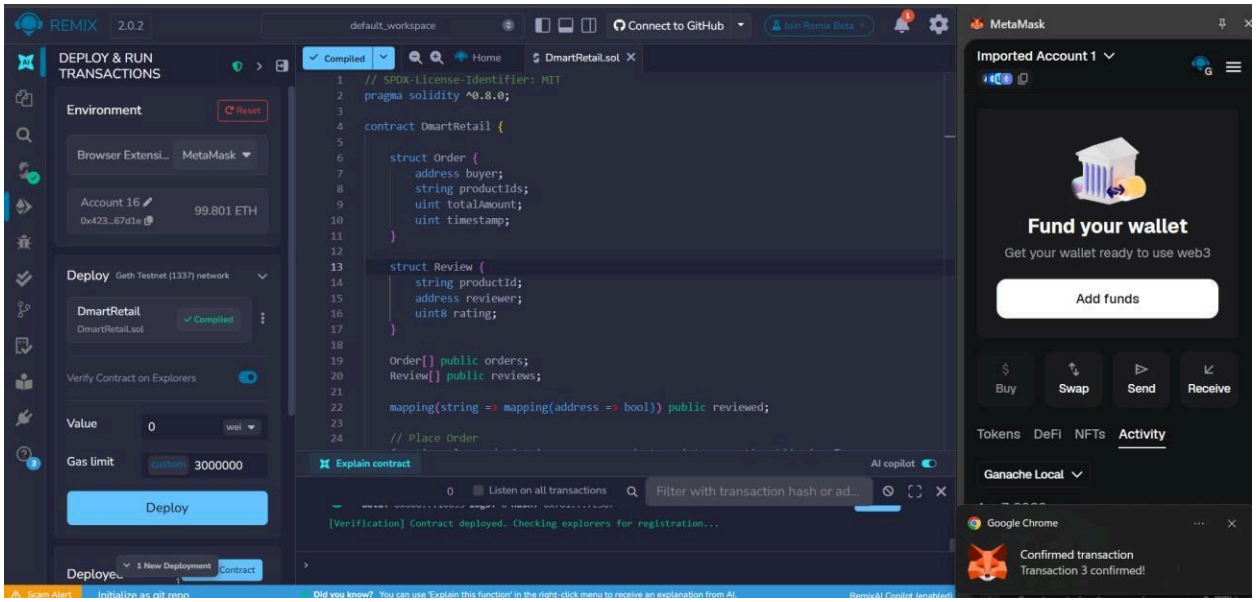
In the Deploy & Run panel, confirm DmartRetail is selected under Contract.

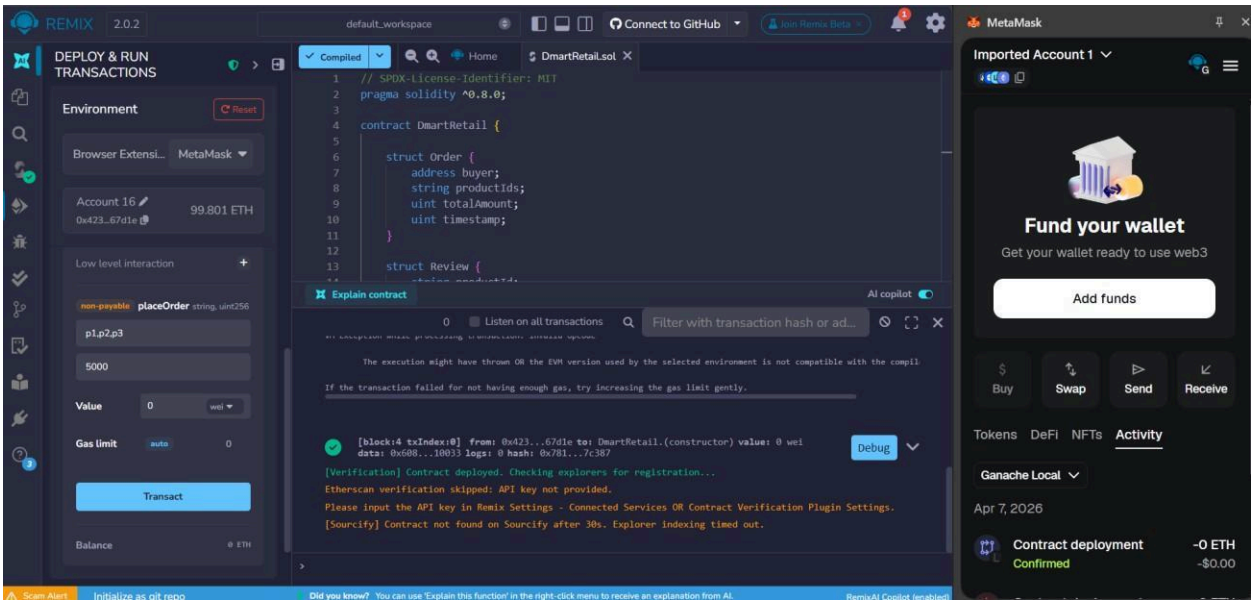
Click Deploy.

MetaMask will open a confirmation popup.



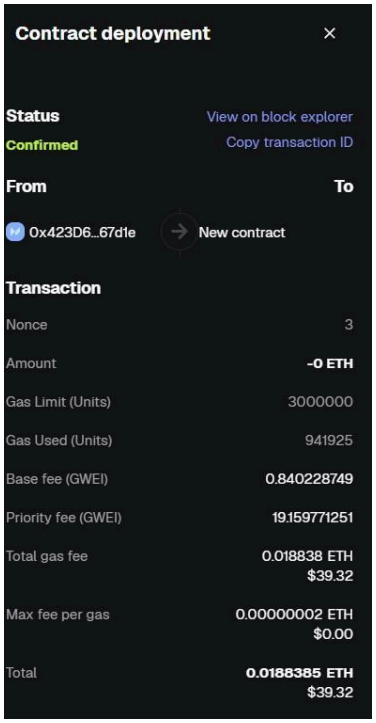

A 'Confirmed transaction' notification from Chrome confirms the deployment was successful



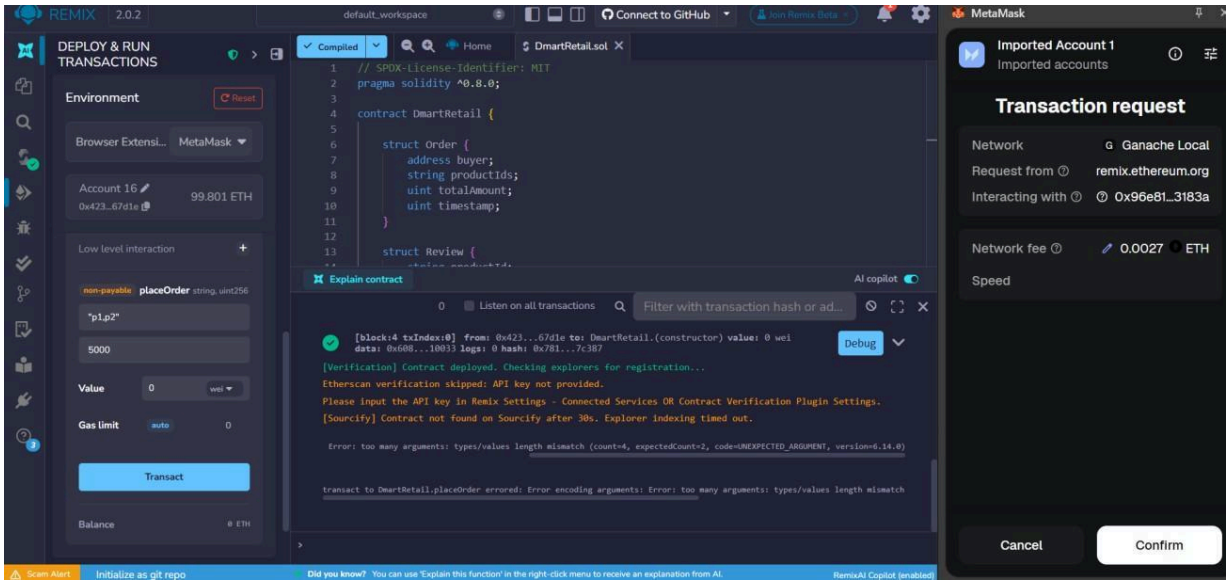

The contract deployment details in MetaMask show:

- Status: Confirmed
- From: 0x423D6…67d1e→ To: New contract
- Gas Used: 941,926 units
- Total gas fee: 0.018838 ETH
- Account balance after: 99.80 ETH

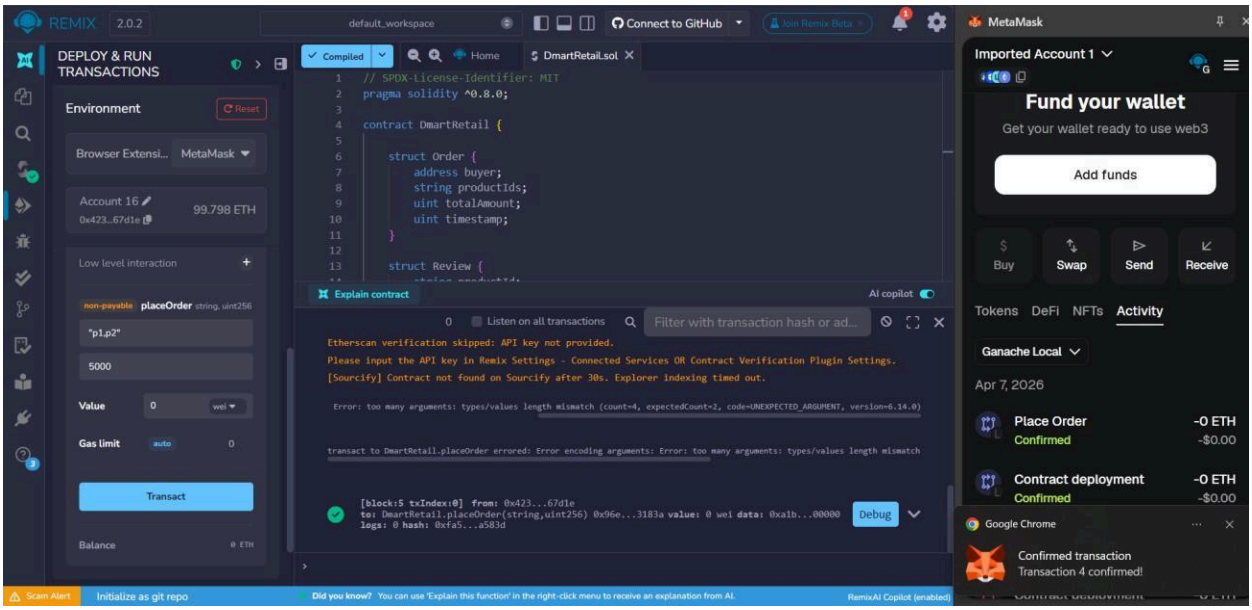



Interact with the Smart Contract After deployment, the contract functions appear in the Remix Deploy & Run panel under Deployed Contracts.

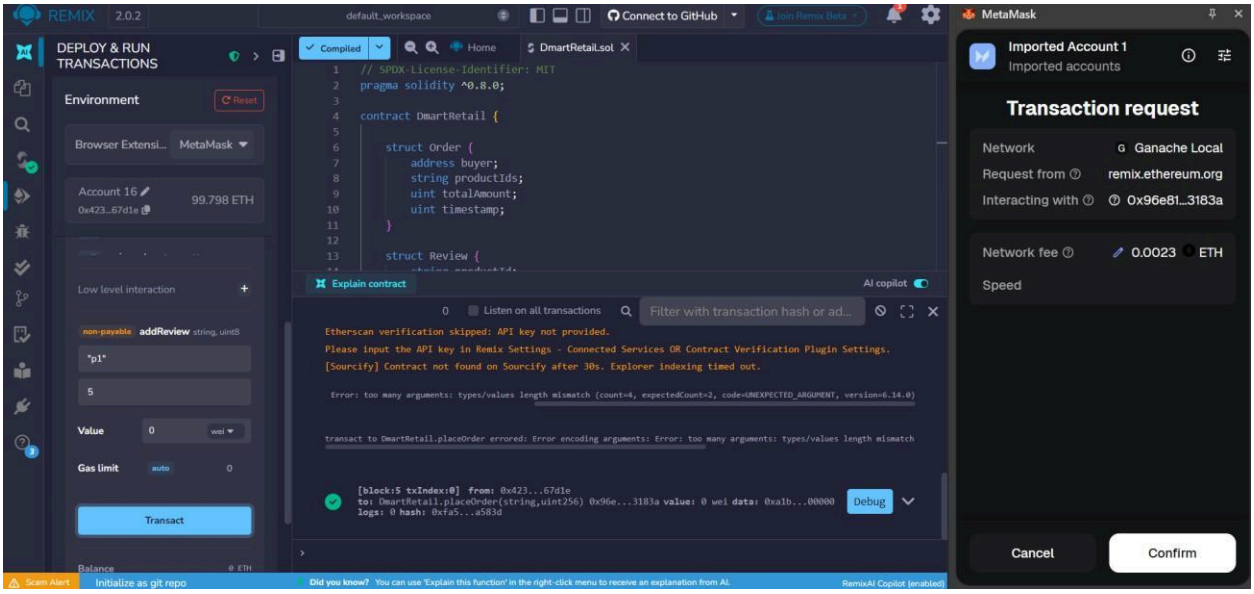
Call placeOrder(p1,p2,5000) where p1,p2 are products and 5000 is the amount.Below screenshot shows the transaction request received by MetaMask from ethereum.



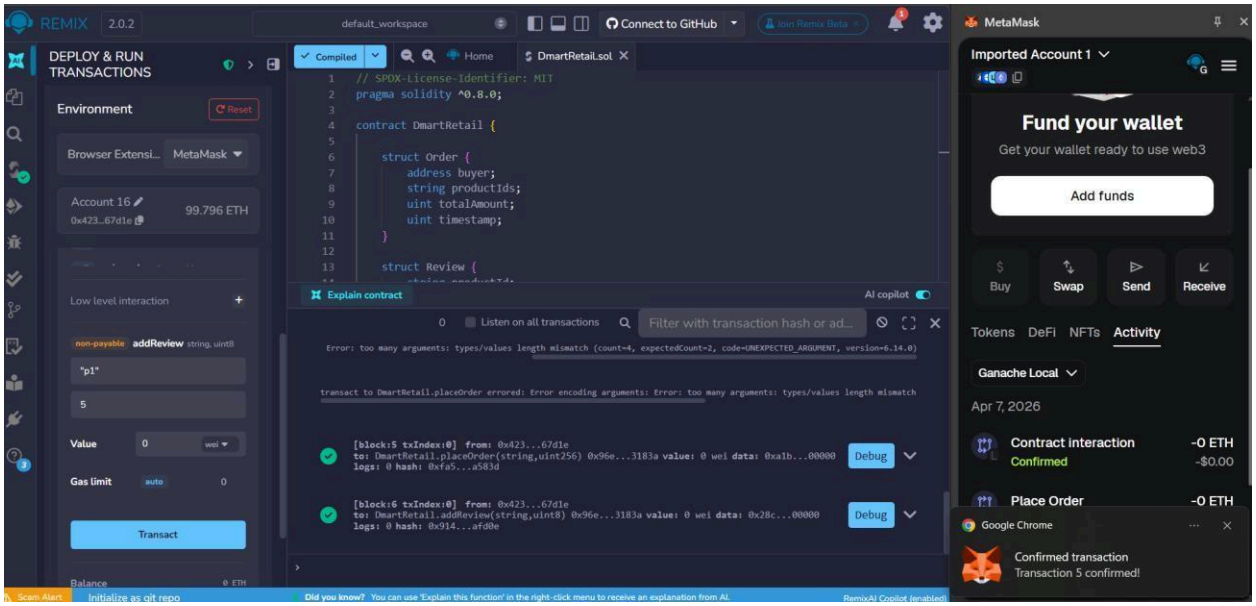
Order Placed is Confirmed.



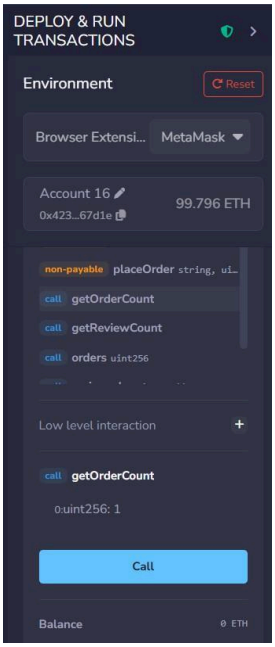

Call addReview where p1 is the product and 5 is the rating(review) given to it.

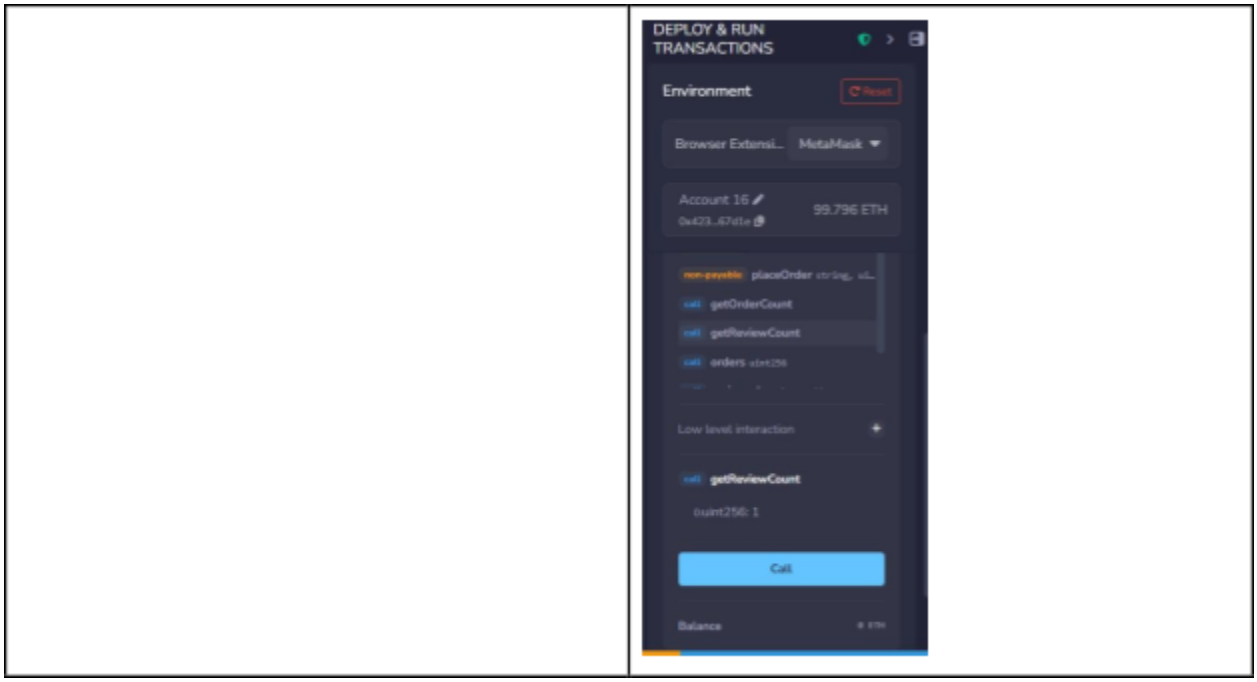



Review Confirmed.




Verify Data using getOrderCount() and getReviewCount()



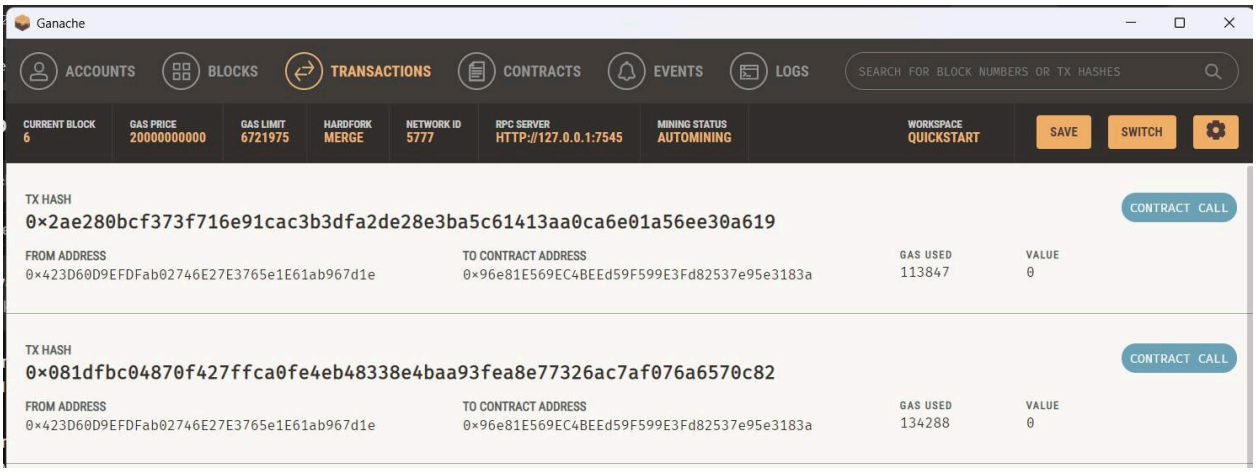



Expected Outcome:

0:

uint256: 1

Output shown in Ganache:




**Conclusion:**
The experiment was successfully completed using Ganache v2.7.1, MetaMask, and Remix IDE. A pharmaceutical supply chain smart contract (PharmaSupplyChain) was written in Solidity, compiled, and deployed to the local Ganache blockchain via

MetaMask. The contract tracked a medicine batch through all four stages of the supply chain — manufacturer, distributor, retailer, and customer — with each stage recorded on-chain. Transaction details were verified in both MetaMask and the Ganache environment, confirming correct gas deduction and block creation